lowThrust — how it works

A Kerbal Space Program mod (C#) that flies a low-thrust vessel to an orbit the player types in — about the same body, a moon, another planet or a planet's moon — within a delta-v / time budget, using the atmosphere where there is one. `lowThrust/mod/`.

1. The parts

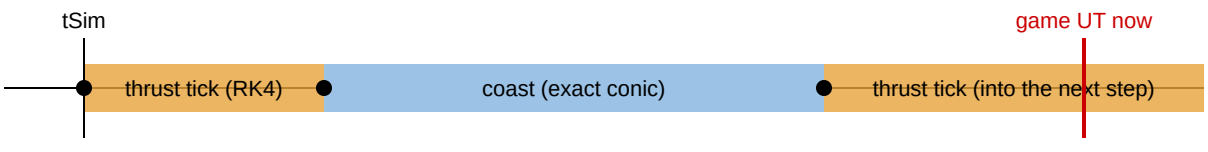
part	what it does
`core/` (no KSP)	The solver. Two-body maths and Lambert (`Orbit`), the within-SOI law (`Guidance`), the stand-in (`Sim`: a point mass, constant thrust, RK4), targeting and capture (`Transfer`), the atmosphere and the drag pass (`Aero`), the phase machine (`Mission`), the dv/time trade (`Tradeoff`), every tunable (`Config`).
`plugin/`	In the game. `Controller` flies the mission, on rails, and hands each pass through the air to the game's physics. `Game` reads the solar system (including each atmosphere's density) and the vessel through KSP's orbital elements, runs the engines on rails, estimates the drag area and sets the warp. `Attitude` turns and holds the vessel with its reaction wheels. `Window` is the toolbar window. `FlightLog` writes one log per flight.
`bridge/` (farm only)	A kRPC service, `LowThrustBridge`, that starts a flight and reports on it, for `tools/ltfly.py`. The mod itself needs no kRPC.
`tools/`, `tests/`	`lowthrust.exe mission|fly` (the stand-in on the command line; `--cda` gives the vessel a drag area so it aerobrakes), and the solver's tests (mono).

2. A flight, from the window

The player picks the body and fills any of periapsis, apoapsis, inclination, LAN and argument of periapsis (blank: free; one apsis alone: circular). Optionally Max Δv and Max time, and the highest warp the autopilot may use.

PLAN flies the whole mission in the stand-in over the trade-off grid (section 5) on worker threads and lists up to 10 options along the frontier. **START**: a Δv limit below `IMPULSIVE_MARGIN` × the impulsive estimate is refused at once; otherwise the engines are lit, SAS goes off, the throttle is shut, the warp limits are relaxed, the drag area is estimated (section 7), the same scan runs (or the clicked option is used), and the first leg's `Mission` is built on a worker thread. **TERMINATE** stops it; warp off, every relaxed limit restored.

3. Every physics step on rails



● a decision (Mission.DoStep) from the state there. Then: orbit written if anything thrust; attitude target set; warp chosen.

Figure 1. One physics step covers whatever game time the warp gives it; commands are decided as they fall due.

Attitude first. `Attitude.Step` turns or holds the vessel toward the target set at the end of the previous step; the thrust axis it leaves, as components against the orbit (radial, along-track, normal), is the axis every thrust tick in this step flies.

Read. If the game's reference body differs from the planner's, the planner takes the game's state, the mission is told (`SoiChange`), and the step ends. Otherwise the state at `tSim` is read from the vessel's orbit elements.

Decide when no command is running: engines and mass refreshed; `Mission.DoStep` returns a throttle, direction and tick, *a pass* (the craft is at the top of the air, inbound: section 7), or the end of the leg. A coast is run ahead in prediction until the mission next wants thrust or a pass, an SOI is near, or `COAST_AHEAD` of a revolution has gone; the span becomes one coast command, the next decision held ready.

Thrust command. It fires only if the thrust axis is within `ALIGN_DEG` when it starts; if not, the vessel coasts only as long as its wheels need to turn there (a bang-bang turn at 0.8 of the weaker of pitch and yaw, plus 5 s), and the mission decides again. A fired command thrusts along the axis as the attitude leaves it each step; RK4 substeps of tick / `THRUST_SUBSTEPS`; every substep draws the propellants (thrust cut to the scarcest), lowers the mass, books the steering cost and checks for an SOI crossing.

Coast command. The exact conic, stopping at an SOI boundary. **Write.** If anything thrust, the vessel's orbit is set through `Orbit.SetOrbit`.

Searches spread over steps. WAIT's moon-push search (up to ~80 trajectory evaluations) runs one evaluation at a time while the step has half its frame budget left, and goes on in the next steps while the coast continues; the warp is held to 5% of the time left to the push point meanwhile. It evaluates exactly what one uninterrupted search would.

Frames, engines, pointing

One right-handed inertial frame per body (x-y the equator, z north); states only through KSP's orbital elements; world space only for pointing, as components against the orbit. Each log opens with a rails check and a frame check. Engines: the lit `ModuleEngines`, vacuum thrust × limiter, Isp at 0 atm, propellant ratios from the part config, drawn through the part's flow rules. Pointing: the target is a burn's direction, or during a coast the next burn's direction where the coast ends. On rails, turning is integrated from the wheels' torque and the measured inertia (EC drawn); once on target it is held exactly and charged ($I\,d\omega/dt + \omega \times I\omega$), let go if the wheels cannot give it. Each thrust substep books the dv lost to the axis being off the command. RCS and gimbal are not used.

4. The warp rate

The highest rails rate index such that one physics step (0.02 s × rate): keeps the autopilot within `FRAME_BUDGET_MS`; draws at most `STEP_RESOURCE_SHARE` of any propellant held; is at most 5 s while the vessel turns; is at most a quarter of the time left to the air (so the mission stops at its top), and 5% of the time left to a pending search's point; is allowed at the altitude (rails limits lowered to the atmosphere top or terrain + `TERRAIN_MARGIN_M`); stays under the game's own clamp before an SOI change (mirrored with margin, and a drop by the game held until the change has passed); is at most `MAX_WARP_INDEX` and the window's max warp. When the vessel is off rails (the game dropped the warp), it goes back on rails no faster than the air ahead allows — or, close above the air and coming down, the pass starts there.

5. Delta-v against time

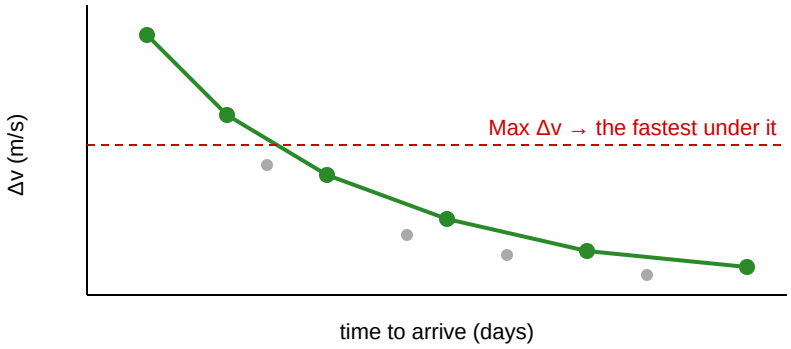
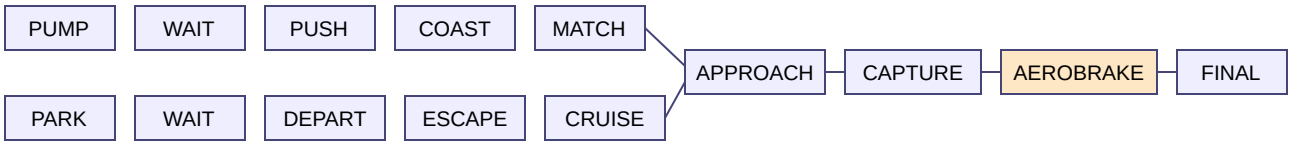


Figure 2. Each point is the whole mission flown in the stand-in at one arc width (and, to another planet, one transfer share). Green: the frontier.

Arc width: the guidance burns in arcs of half-width acos(η) round each apsis; `PLAN_GRID` values. **Transfer share** (to another planet): the cheapest Lambert window arriving within `TRANSFER_TIME_SHARE` of the cheapest window's wait plus flight, at each of `TRANSFER_SHARES`. Max Δv → the fastest within it; max days → the cheapest within both; none → the fastest within `BALANCE_DV_SHARE` of the cheapest; a clicked option → that one; nothing → refused with what exists. The stand-in plans with the vessel's drag area (aerobraking included, heating limit `AERO_HEAT_PLAN`).

6. The mission's phases



to a moon (top), to a planet (bottom); MATCH only when the arrival is too fast to capture. AEROBRAKE only where there is air; within one SOI: FINAL (→ AEROBRAKE → FINAL). Anywhere: TRIM (phasing to miss a moon), the floor guard.

Figure 3. The phases and what follows what; each is re-decided every tick from the current state.

phase	flies	ends when
PUMP / PARK / FINAL	`Guidance`: prograde/retrograde arcs on a held apse line, each fixing the opposite apsis (outward first); plane change at the nodes on the biggest orbit; arcs widened until the changes left fit `ARC_PASSES_MAX` passes, or `ETA_WIDE_MIN` while crossing another moon's band; each arc's burn capped so the next revolution meets no moon; a phasing trim once a revolution if one would. FINAL at a body with air and an apoapsis above the final one hands over to AEROBRAKE.	every targeted element within tolerance → WAIT, or done (FINAL)
WAIT (moon)	Coasts. At each push point (periapsis, or every 30° if no dearer than `PUSH_OFF_PERIAPSIS`) it solves the prograde push whose coast enters the moon's SOI on the aim (the finite burn; spread over steps, section 3); skips a pass that meets another moon or arrives too fast to capture, and remembers when the planned encounter is.	half the push before its point → PUSH
PUSH	Prograde, re-solved round its value each tick, looking at least 1.2 × as far ahead as the encounter WAIT planned (the post-push revolution's 0.75 can end just short of it)	< `CORRECT_DV` left → COAST
WAIT / DEPART (planet)	The window from the transfer share; waits for the periapsis pass nearest it (checked along the burn DEPART will fly), then burns prograde to the window's v∞	v∞ reached or `DEPART_ARC_DEG` past periapsis → ESCAPE (or WAIT if still bound)

COAST / CRUISE	B-plane correction toward the capture periapsis; Lambert re-target when the miss is too big to linearise	SOI entry → APPROACH; MATCH due → MATCH
MATCH	ZEM/ZEV guidance about the parent onto a point outside the SOI, arriving at `ARRIVE_MARGIN` of the fastest capturable speed	matched, `MATCH_END_S` before, or given up
APPROACH	On SOI entry at a body with air, the capture periapsis is moved into the air (section 7). Nudges the periapsis onto it; dodges a moon on the way in	the capture law says burn → CAPTURE
CAPTURE	Burns centred on periapsis if it fits one pass; otherwise starts once the energy to shed reaches `CAPTURE_PATH_SHARE` of what full thrust removes on the path left, and throttles at that ratio. The energy the coming pass through the air sheds at its low end (drag ÷ margin) is left to it. Direction: retrograde above the middle of the periapsis band, its pull on the periapsis faded out smoothly toward the aim (5% above the floor at least); below the aim a lift along the periapsis gradient closes the error over the time left to periapsis (at most half the thrust; under the floor, ramped in over 5% of it, all of it). The direction never jumps between ticks.	bound below the capture apoapsis (or the moon-clear ceiling past periapsis) → AEROBRAKE where there is air, else FINAL
AEROBRAKE	Section 7	a pass would gain too little → periapsis lifted out → FINAL

A thrusting tick changes the velocity by at most `THRUST_DV_SHARE`; ticks are uniform in angle and short enough not to jump an SOI boundary — or the top of the air.

7. Aerobraking

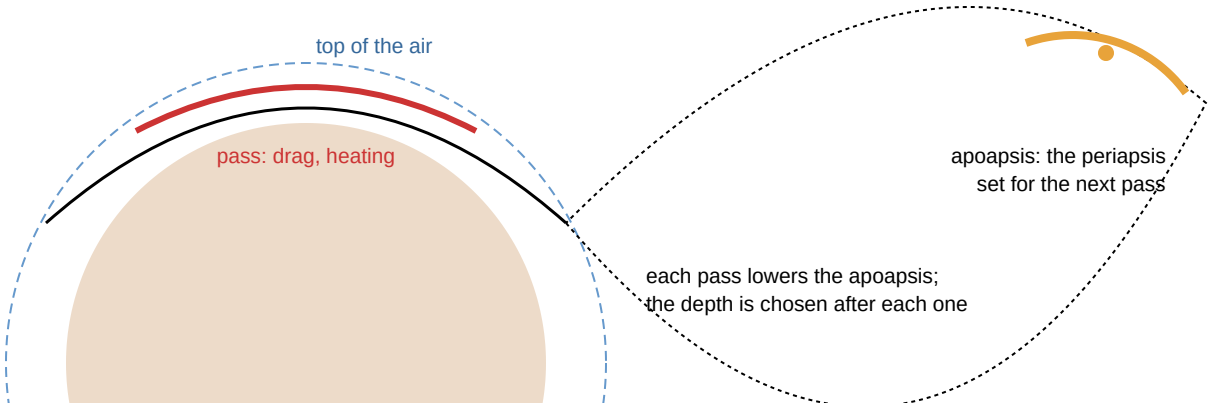


Figure 4. AEROBRAKE: passes through the air at periapsis, the periapsis for the next one set by a burn round apoapsis.

The model (`Aero`)

- **Air**: density every 1 km from sea level to the top (0 there), log-linear between. In the game it is sampled from the body (`GetDensity(GetPressure, GetTemperature)` at the base temperature); the stand-in's tables were sampled from the game. The air turns with the body.
- **A pass** (`Aero.Pass`): RK4 from the top down and out — gravity plus drag q × cdA × k / m against the turning air, no thrust; step 500 m of path. It reports the energy shed, the drag Δv, the heating index H = $\sqrt{\rho}\, v^3 / 10^6$ (KSP's high-Mach convective coefficient less its constant) at its peak and integrated (the load), the peak deceleration and the lowest altitude.
- **The deepest pass** (`Aero.Deepest`, bisection on the periapsis): the pass flown at `AERO_DRAG_MARGIN` × the drag and the density must stay above the highest terrain + `AERO_TERRAIN_MARGIN_M` and `AERO_MIN_ALT_SHARE` of the air's depth; keep H and the load under the limits learned; decelerate at most

`AERO_ACCEL_MAX`; leave an apoapsis at or above the final one; and leave a periapsis the engine can lift to `AERO_LIFT_MARGIN_M` above the air within ±60° of the next apoapsis.

Capture

On SOI entry the capture periapsis becomes the deepest pass safe both for the arrival (its energy) and for the orbit the capture burn will leave (apoapsis the capture's). The capture law (section 6) leaves to the pass the energy it sheds at the low end.

AEROBRAKE

- After each pass (and at the start), the depth for the next: the deepest, or, if its next revolution meets a moon (checked at the modelled drag and at the margin above and below), the first shallower one in tenths of the way to the top that does not; if none does but the orbit as it is stays clear a revolution more, the periapsis is kept out of the air for one revolution.
- The guidance sets it: target apoapsis free, periapsis the depth, the target plane carried (so the plane change happens at the high apoapsis); the burn is the arc round apoapsis.
- Ends when a pass would lower the apoapsis by less than `AERO_MIN_GAIN` of what is left above the final one — not while the heating limit is still growing. Then the periapsis is lifted out of the air in the arc round apoapsis, widened (up to ±60°) to fit the lift, and FINAL shapes the orbit.

Learning, in the game

- The heating allowed starts at `AERO_HEAT_FIRST`. After each pass, every part's rise of max(skin ÷ skin maximum, interior ÷ maximum) from its value at the pass's start is extrapolated linearly to `AERO_HEAT_TARGET` of the maximum; the smallest factor over the parts (at most `AERO_HEAT_GROWTH`, at least 0.2) times the pass's modelled H and load are the limits for the next (raised only if the factor is above 1, lowered if below).
- The drag scale k = the energy the pass shed ÷ the model's at k = 1 from the same entry state. The drag area starts from the parts' drag cubes (Mach 6, flow on the held nose).
- A pass not flown in the air (the vessel came off rails already past it) teaches nothing. The stand-in plans at `AERO_HEAT_PLAN` and learns nothing.

Flying a pass

The mission stops at the top of the air, inbound (the warp's steps shrink to it). The controller drops the warp; once the game has the vessel off rails it sets physics warp `AERO_PHYSICS_WARP` and SAS to `AERO_HOLD`, and measures every step: H, the load, the dynamic pressure, the drag (the parts' `dragScalar`) and so the drag area, the deceleration, every part's temperatures. Out of the air, climbing, the temperatures are watched until none has risen for 20 s (at least 10 s, at most `AERO_SOAK_S`); then SAS goes off, the orbit is read back, the mission is told what the pass shed and how hot it ran, and the vessel goes back on rails. Nothing thrusts in a pass.

8. The log

`GameData/LowThrust/PluginData/logs/flight-<date>.log`: the `config:` line and defaults fingerprint, the rails check, the target, the vessel, the frame check, the propellants, the drag area (where passes may be flown), the `plan:`; the mission's events and telemetry at most once per `LOG_INTERVAL_UT`; for each pass its plan (depth, apoapsis before and after, drag Δv, heating and limit, load, deceleration), its entry (periapsis, speed, how far the game was ahead and at what warp), a line every 30 game seconds in the air, and its result (lowest altitude, apoapsis change, energy shed against the model, drag scale, heating limits, measured drag area and the hottest part's rise); at the end the result, wall and game seconds by phase, the

autopilot's time per step, thrust ticks fired and not aligned, propellant shortfalls, the wheels' EC, and the `steering:` line.